Export Kode UTS - Kelompok HHH

Dani Setiawan (240712771), Isidorus Ambar Wicaksono (240712775), Johannes Calvin (240712781)

1. KelpHHH_dataset_EDA.ipynb (sel kode)

```
# --- sel 1 ---
import re, json
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
pd.set_option("display.max_colwidth", 120)
DATA = Path("../dataset")
RAW = DATA / "raw"
ORIG = DATA / "KelpHHH_dataset_original.csv"
CLEAN = DATA / "KelpHHH_dataset_clean.csv"
# Parameter cleaning dan sampling (ubah di sini bila kesepakatan kelompok/dosen berubah)
MIN_WORDS = 3          # teks dengan kurang dari 3 kata dianggap terlalu pendek
N_ANNOTATE = 1000      # jumlah teks yang diajukan ke anotasi (target akhir minimal 800 setelah Ignore)
SEED = 42
# --- sel 2 ---
tr = pd.read_csv(RAW / "train.txt");  tr["source_split"] = "train"
va = pd.read_csv(RAW / "validation.txt"); va["source_split"] = "validation"
orig = pd.concat([tr, va], ignore_index=True)
orig.insert(0, "id", range(len(orig)))      # id baris = penelusuran ke data asli
orig.to_csv(ORIG, index=False)
print("Train:", tr.shape, "| Validation:", va.shape, "| Gabungan:", orig.shape)
orig.head()
# --- sel 3 ---
print(orig.dtypes, "\n")
print("Missing values per kolom:\n", orig.isna().sum(), "\n")
print("Distribusi label asli:\n", orig["label"].value_counts(), "\n")
print("Distribusi bintang:\n", orig["stars"].value_counts().sort_index())
fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
sns.countplot(data=orig, x="label", ax=ax[0]); ax[0].set_title("Label asli (sentimen)")
sns.countplot(data=orig, x="stars", ax=ax[1]); ax[1].set_title("Distribusi bintang")
plt.tight_layout(); plt.show()
# --- sel 4 ---
raw = orig["text"].astype(str)
norm_ws = raw.str.replace(r"\s+", " ", regex=True).str.strip()
key = norm_ws.str.lower().str.replace(r"[^\w\s]", "", regex=True)
n_words = norm_ws.str.split().str.len()
quality = pd.Series({
    "Total baris": len(orig),
    "Teks kosong / hanya whitespace": int((norm_ws == "").sum()),
    "Teks tanpa huruf (emoji/simbol/angka saja)": int((~norm_ws.str.contains(r"[A-Za-z]")).sum()),
    "Teks duplikat (setelah normalisasi)": int(key.duplicated().sum()),
    f"Teks < {MIN_WORDS} kata": int((n_words < MIN_WORDS).sum()),
    "Mengandung markup HTML/URL": int(norm_ws.str.contains(r"<[^>]+>|https?://|www\.").sum()),
    "Mengandung karakter encoding rusak (U+FFFD)": int(raw.str.contains(chr(0xFFFD), regex=False).sum()),
})
quality.to_frame("jumlah")
# --- sel 5 ---
ID_MARK = r"\b(yang|yg|dan|di|ke|dari|untuk|saya|sy|aku|gak|ga|nggak|tidak|tdk|sudah|udah|sdh|banget|bgt|aplikasi|barang|brg|pen
    giriman|kirim|bayar|belanja|sangat|juga|jg|tapi|tp|dengan|dgn|ini|itu|ada|bisa|bs|lebih|sering|mudah|cepat|cepet|murah|harga
    |promo|gratis|ongkir|produk|pesanan|saja|aja|nya|sih|kok|lagi|masih|belum|mau|karena|krn|terus|trs|dalam|atau|sekali|bagus|p
    akai|pake|lama|lambat|kurir)\b"
EN_MARK = r"\b(the|is|are|was|and|very|this|that|it|app|good|great|love|easy|use|not|but|for|with|have|my|so|can|you|to|of|in|on
    )\b"
df = orig.rename(columns={"text": "text_raw"}).copy()
log = [("Data asli", len(df))]
df["text"] = df["text_raw"].astype(str).str.replace(r"\s+", " ", regex=True).str.strip()
df = df[df["text"] != ""];                                  log.append(("Buang teks kosong", len(df)))
df = df[df["text"].str.contains(r"[A-Za-z]")];              log.append(("Buang teks tanpa huruf", len(df)))
df = df[~(df["text"].str.contains(r"<[^>]+>|https?://|www\.") | df["text"].str.contains(chr(0xFFFD), regex=False))]; log.append(
    ("Buang markup/URL/encoding rusak", len(df)))
df["dedup_key"] = df["text"].str.lower().str.replace(r"[^\w\s]", "", regex=True)
df = df.drop_duplicates("dedup_key", keep="first");         log.append(("Deduplikasi (case/tanda baca diabaikan)", len(df)))
df["n_words"] = df["text"].str.split().str.len()
df = df[df["n_words"] >= MIN_WORDS];                        log.append((f"Buang teks < {MIN_WORDS} kata", len(df)))
low = df["text"].str.lower()
df["_id"] = low.str.count(ID_MARK); df["_en"] = low.str.count(EN_MARK)
df = df[~((df["_id"] == 0) & (df["_en"] > 0))];             log.append(("Buang teks Inggris murni (heuristik)", len(df)))
df = df.drop(columns=["dedup_key", "_id", "_en"]).reset_index(drop=True)
cleaning_log = pd.DataFrame(log, columns=["langkah", "baris_tersisa"])
cleaning_log["dibuang"] = (-cleaning_log["baris_tersisa"].diff()).fillna(0).astype(int)
cleaning_log
# --- sel 6 ---
print("Contoh teks yang dipertahankan:")
display(df[["id", "text", "label", "stars"]].sample(8, random_state=SEED))
print("Contoh perubahan whitespace (raw -> clean):")
chg = df[df["text_raw"].astype(str) != df["text"]].head(3)
for _, r in chg.iterrows():
    print(repr(r["text_raw"])); print(" ->", repr(r["text"])); print()
```

```
# --- sel 7 ---
CUES = {
    "PENGIRIMAN": r"kurir|kirim|pengiriman|paket|packing|sampai|sampe|ekspedisi|jne|jnt|sicepat|diterima|telat",
    "LAYANAN_PELANGGAN": r"\bcs\b|customer service|costumer|refund|retur|komplain|seller|admin|respon|balas|hubungi|pengembalian
    ",
    "APLIKASI_PERFORMA": r"error|bug|lemot|lambat|\blag\b|loading|force close|crash|hang|nge-?lag|keluar sendiri|not responding|
    gagal install|gak bisa dibuka",
    "USABILITY": r"tampilan|mudah|susah|ribet|menu|fitur|cari|pencarian|navigasi|simpel|praktis|login|user friendly|bingung",
    "PEMBAYARAN": r"bayar|pembayaran|checkout|transaksi|saldo|e-?wallet|ovo|gopay|dana|virtual account|\bva\b|kartu kredit|cicil
    an|transfer",
    "PROMO_HARGA": r"promo|diskon|voucher|cashback|harga|mahal|murah|gratis|potongan|hemat",
}
for lab, pat in CUES.items():
    df["cue_" + lab] = df["text"].str.lower().str.contains(pat, regex=True)
cue_cols = ["cue_" + l for l in CUES]
df["n_cues"] = df[cue_cols].sum(axis=1)
df["has_aspect_cue"] = df["n_cues"] > 0
print("Teks bersih:", len(df), "| dengan minimal satu cue aspek:", int(df["has_aspect_cue"].sum()))
df[cue_cols].sum().rename(lambda c: c.replace("cue_", "")).to_frame("jumlah_teks_dengan_cue")
# --- sel 8 ---
# Sampel anotasi: dari teks ber-cue, stratifikasi mengikuti proporsi label asli (pos/neg) agar ulasan negatif tidak hilang
pool = df[df["has_aspect_cue"]]
n = min(N_ANNOTATE, len(pool))
frac = n / len(pool)
sample_ids = (pool.groupby("label", group_keys=False)
                  .apply(lambda g: g.sample(frac=frac, random_state=SEED))["id"])
df["in_annotation_sample"] = df["id"].isin(sample_ids)
print("Teks diajukan ke anotasi:", int(df["in_annotation_sample"].sum()))
print(df[df["in_annotation_sample"]]["label"].value_counts().to_string())
# --- sel 9 ---
final_cols = ["id", "text", "text_raw", "label", "stars", "source_split", "n_words",
              "has_aspect_cue", "n_cues", "in_annotation_sample"]
assert df["text"].is_unique, "Teks tidak unik"
df[final_cols].to_csv(CLEAN, index=False)
print("Disimpan:", CLEAN, df[final_cols].shape)
# --- sel 10 ---
fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
sns.histplot(df["n_words"].clip(upper=60), bins=30, ax=ax[0]); ax[0].set_title("Panjang teks (kata, dipotong di 60)")
sns.histplot(df["text"].str.len().clip(upper=400), bins=30, ax=ax[1]); ax[1].set_title("Panjang teks (karakter, dipotong di 400)
    ")
plt.tight_layout(); plt.show()
print(df["n_words"].describe().round(1).to_string())
# --- sel 11 ---
counts = df[cue_cols].sum().rename(lambda c: c.replace("cue_", "")).sort_values()
ax = counts.plot.barh(figsize=(7, 3.5), title="Teks dengan cue per aspek (perkiraan awal, bukan label)")
plt.tight_layout(); plt.show()
print("Jumlah cue aspek per teks:\n", df["n_cues"].value_counts().sort_index().to_string())
# --- sel 12 ---
co = df[cue_cols].astype(int).T.dot(df[cue_cols].astype(int))
co.index = co.columns = [c.replace("cue_", "") for c in co.columns]
plt.figure(figsize=(7, 5.5))
sns.heatmap(co, annot=True, fmt="d", cmap="Blues")
plt.title("Ko-okurensi cue aspek (perkiraan awal)")
plt.tight_layout(); plt.show()
# --- sel 13 ---
# Contoh evidence dari korpus untuk tiap aspek (bahan justifikasi label dan contoh guideline)
for lab in CUES:
    ex = df[df["cue_" + lab] & (df["n_words"].between(6, 25))].sample(3, random_state=SEED)["text"].tolist()
    print(f"[{lab}]")
    for e in ex: print("  -", e)
    print()
# --- sel 14 ---
import itertools
from collections import Counter
LABELS = list(CUES.keys())
ann_path = DATA / "KelpHHH_dataset_annotation.jsonl"
if not ann_path.exists():
    ann_path = DATA / "KelpHHH_gold_overlap.jsonl"
rows = [json.loads(l) for l in open(ann_path, encoding="utf-8") if l.strip()]
ann = pd.DataFrame(rows)
ann["n_words"] = ann["text"].str.split().str.len()
ann["n_labels"] = ann["labels"].str.len()
ann["n_entities"] = ann["entities"].str.len()
for r in rows:   # validasi offset span terhadap teks
    for e in r["entities"]:
        assert r["text"][e["start"]:e["end"]] == e["text"], (r["id"], e)
print("Sumber:", ann_path.name, "| teks:", len(ann), "| total span:", int(ann["n_entities"].sum()))
# --- sel 15 ---
fig, ax = plt.subplots(2, 3, figsize=(15, 7))
sns.histplot(ann["n_words"].clip(upper=80), bins=25, ax=ax[0,0]); ax[0,0].set_title("Panjang teks (kata)")
cnt = pd.Series(Counter(l for ls in ann["labels"] for l in ls)).reindex(LABELS).fillna(0)
cnt.sort_values().plot.barh(ax=ax[0,1], title="Distribusi label (jumlah teks)")
ann["n_labels"].value_counts().sort_index().plot.bar(ax=ax[0,2], title="Jumlah label per teks")
ent = pd.DataFrame([{"label": e["label"], "n_tok": len(e["text"].split()), "text": e["text"].lower()} for es in ann["entities"]
    for e in es])
ent["label"].value_counts().reindex(LABELS).fillna(0).sort_values().plot.barh(ax=ax[1,0], title="Jumlah span per label")
ann["n_entities"].value_counts().sort_index().plot.bar(ax=ax[1,1], title="Jumlah span per teks")
sns.histplot(ent["n_tok"].clip(upper=12), bins=12, discrete=True, ax=ax[1,2]); ax[1,2].set_title("Panjang span (token)")
plt.tight_layout(); plt.show()
print("Imbalance ratio label (maks/min): %.2f" % (cnt.max() / cnt[cnt > 0].min()))
```

```
print("Imbalance ratio span  (maks/min): %.2f" % (ent["label"].value_counts().max() / ent["label"].value_counts().min()))
# --- sel 16 ---
B = pd.DataFrame({l: ann["labels"].apply(lambda x, l=l: int(l in x)) for l in LABELS})
co = B.T.dot(B)
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
sns.heatmap(co, annot=True, fmt="d", cmap="Blues", ax=ax[0]); ax[0].set_title("Co-occurrence label")
sns.heatmap(B.corr(), annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1, ax=ax[1]); ax[1].set_title("Korelasi (phi) antarl
    abel")
plt.tight_layout(); plt.show()
combo = ann["labels"].apply(lambda x: " + ".join(sorted(x)) or "(kosong)").value_counts().head(10)
combo.to_frame("jumlah")
# --- sel 17 ---
# Dominasi tag O dan contoh span paling sering (token = pemisah spasi)
tot_tok = int(ann["n_words"].sum()); span_tok = int(ent["n_tok"].sum())
print("Token dalam span: %d dari %d (%.1f%%); tag O sekitar %.1f%%" % (span_tok, tot_tok, 100*span_tok/tot_tok, 100*(1-span_tok/
    tot_tok)))
top = ent.groupby(["label", "text"]).size().reset_index(name="n").sort_values("n", ascending=False)
for lab in LABELS:
    print(lab, "->", ", ".join(f"{t} ({n})" for t, n in top[top.label == lab].head(5)[["text", "n"]].values))
```

2. KelpHHH_dataset_explorer.py

```
"""KelpHHH Dataset Explorer (UTS PBA 2026/2027).
Menjalankan:  streamlit run KelpHHH_dataset_explorer.py
Data dibaca dari ../dataset/KelpHHH_dataset_annotation.jsonl (jika ada), jika tidak ada
dipakai ../dataset/KelpHHH_gold_overlap.jsonl. Dapat juga diunggah lewat sidebar.
Format baris JSONL:
{"id": 1, "text": "...", "labels": ["PENGIRIMAN"],
 "entities": [{"start": 0, "end": 5, "label": "PENGIRIMAN", "text": "Kurir"}]}
"""
import html
import itertools
import json
from collections import Counter
from pathlib import Path
import pandas as pd
import streamlit as st
LABELS = ["PENGIRIMAN", "LAYANAN_PELANGGAN", "APLIKASI_PERFORMA", "USABILITY", "PEMBAYARAN", "PROMO_HARGA"]
COLORS = {
    "PENGIRIMAN": "#ffd166",
    "LAYANAN_PELANGGAN": "#ef9a9a",
    "APLIKASI_PERFORMA": "#ce93d8",
    "USABILITY": "#90caf9",
    "PEMBAYARAN": "#a5d6a7",
    "PROMO_HARGA": "#ffab91",
}
HERE = Path(__file__).resolve().parent
CANDIDATES = [
    HERE.parent / "dataset" / "KelpHHH_dataset_annotation.jsonl",
    HERE.parent / "dataset" / "KelpHHH_gold_overlap.jsonl",
    HERE / "KelpHHH_dataset_annotation.jsonl",
]
st.set_page_config(page_title="KelpHHH Dataset Explorer", layout="wide")
# ---------------------------------------------------------------- data
def parse_jsonl(lines):
    rows = []
    for line in lines:
        line = line.strip()
        if not line:
            continue
        r = json.loads(line)
        r.setdefault("labels", [])
        r.setdefault("entities", [])
        rows.append(r)
    return rows
@st.cache_data(show_spinner=False)
def load_path(path: str):
    with open(path, encoding="utf-8") as f:
        return parse_jsonl(f)
def get_rows():
    up = st.sidebar.file_uploader("Unggah file anotasi (.jsonl)", type=["jsonl", "json"])
    if up is not None:
        return parse_jsonl(up.getvalue().decode("utf-8").splitlines()), f"upload: {up.name}"
    for p in CANDIDATES:
        if p.exists():
            return load_path(str(p)), p.name
    return [], ""
def render_highlight(text, entities):
    """Highlight span; mendukung overlap antarlabel (label ditumpuk sebagai badge)."""
    n = len(text)
    cover = [[] for _ in range(n)]
    for e in entities:
        for i in range(max(0, e["start"]), min(n, e["end"])):
            cover[i].append(e["label"])
    out, i = [], 0
    while i < n:
        labs = tuple(sorted(set(cover[i])))
        j = i
        while j < n and tuple(sorted(set(cover[j]))) == labs:
            j += 1
        chunk = html.escape(text[i:j])
        if labs:
            if len(labs) == 1:
                bg = COLORS.get(labs[0], "#ddd")
            else:
                c1, c2 = COLORS.get(labs[0], "#ddd"), COLORS.get(labs[1], "#ddd")
                bg = f"linear-gradient(90deg,{c1},{c2})"
            tag = "+".join(l.split("_")[0] for l in labs)
            out.append(
                f'<mark style="background:{bg};padding:1px 3px;border-radius:4px;color:#111">{chunk}'
                f'<sup style="font-size:0.6em;font-weight:700;margin-left:2px">{tag}</sup></mark>'
            )
        else:
            out.append(chunk)
        i = j
    return "".join(out)
rows, source = get_rows()
st.title("KelpHHH Dataset Explorer")
st.caption("Ulasan aplikasi e-commerce (Blibli, Google Play) | Multilabel aspek + span evidence")
```

```
if not rows:
    st.warning("Belum ada data. Letakkan KelpHHH_dataset_annotation.jsonl di folder ../dataset atau unggah lewat sidebar.")
    st.stop()
st.sidebar.success(f"Sumber data: {source} ({len(rows)} teks)")
df = pd.DataFrame(
    {
        "id": [r.get("id", i) for i, r in enumerate(rows)],
        "text": [r["text"] for r in rows],
        "labels": [r["labels"] for r in rows],
        "entities": [r["entities"] for r in rows],
    }
)
df["n_chars"] = df["text"].str.len()
df["n_tokens"] = df["text"].str.split().str.len()
df["n_labels"] = df["labels"].str.len()
df["n_entities"] = df["entities"].str.len()
df["label_combo"] = df["labels"].apply(lambda x: " + ".join(sorted(x)) if x else "(tanpa label)")
all_labels = LABELS + sorted({l for ls in df["labels"] for l in ls} - set(LABELS))
page = st.sidebar.radio("Halaman", ["Overview", "Browse Data", "Multilabel Analysis", "NER Analysis", "Filter / Search"])
# ---------------------------------------------------------------- Overview
if page == "Overview":
    c = st.columns(4)
    c[0].metric("Jumlah teks", len(df))
    c[1].metric("Rata-rata kata/teks", f"{df.n_tokens.mean():.1f}")
    c[2].metric("Rata-rata label/teks", f"{df.n_labels.mean():.2f}")
    c[3].metric("Total span", int(df.n_entities.sum()))
    st.subheader("Taxonomy label")
    desc = {
        "PENGIRIMAN": "Kecepatan/ketepatan kirim, kurir, ekspedisi, kondisi kemasan paket.",
        "LAYANAN_PELANGGAN": "Respon dan penanganan CS/seller, refund, retur, komplain.",
        "APLIKASI_PERFORMA": "Kestabilan teknis: error, bug, lemot, force close, loading.",
        "USABILITY": "Kemudahan memakai aplikasi: tampilan, navigasi, pencarian, fitur.",
        "PEMBAYARAN": "Proses bayar/checkout, metode bayar, cicilan, saldo, kegagalan transaksi.",
        "PROMO_HARGA": "Diskon, voucher, gratis ongkir, cashback, harga.",
    }
    cnt = Counter(l for ls in df.labels for l in ls)
    ecnt = Counter(e["label"] for es in df.entities for e in es)
    st.dataframe(
        pd.DataFrame(
            {
                "Label": all_labels,
                "Definisi": [desc.get(l, "") for l in all_labels],
                "Jumlah teks": [cnt.get(l, 0) for l in all_labels],
                "Jumlah span": [ecnt.get(l, 0) for l in all_labels],
            }
        ),
        use_container_width=True,
        hide_index=True,
    )
    st.markdown(
        "Label Skema A (dokumen) dan label span Skema B memakai **satu set 6 label yang sama**. "
        "Sentimen tidak dianotasi. Ulasan tanpa aspek di-*Ignore* dan tidak masuk dataset."
    )
    st.subheader("Distribusi panjang teks")
    st.bar_chart(df["n_tokens"].clip(upper=60).value_counts().sort_index())
# ---------------------------------------------------------------- Browse
elif page == "Browse Data":
    per = st.sidebar.slider("Teks per halaman", 5, 50, 10)
    pages = max(1, (len(df) + per - 1) // per)
    pg = st.sidebar.number_input("Halaman", 1, pages, 1)
    sub = df.iloc[(pg - 1) * per : pg * per]
    st.write(f"Menampilkan {len(sub)} dari {len(df)} teks (halaman {pg}/{pages})")
    for _, r in sub.iterrows():
        with st.container(border=True):
            st.markdown(f"**ID {r['id']}**  |  label: `{', '.join(r['labels']) or '-'}`")
            st.markdown(render_highlight(r["text"], r["entities"]), unsafe_allow_html=True)
            if r["entities"]:
                st.caption(" ; ".join(f"{e['text']} = {e['label']} [{e['start']}:{e['end']}]" for e in r["entities"]))
# ---------------------------------------------------------------- Multilabel
elif page == "Multilabel Analysis":
    cnt = Counter(l for ls in df.labels for l in ls)
    dist = pd.Series({l: cnt.get(l, 0) for l in all_labels})
    st.subheader("Distribusi label")
    st.bar_chart(dist)
    mx, mn = dist.max(), dist[dist > 0].min() if (dist > 0).any() else 0
    st.write(f"**Imbalance ratio** (label terbanyak / tersedikit): {mx / mn:.2f}" if mn else "-")
    st.subheader("Jumlah label per teks")
    st.bar_chart(df["n_labels"].value_counts().sort_index())
    st.subheader("Kombinasi label terbanyak")
    combo = df["label_combo"].value_counts().head(15).rename_axis("kombinasi").reset_index(name="jumlah")
    st.dataframe(combo, use_container_width=True, hide_index=True)
    st.subheader("Co-occurrence label")
    M = pd.DataFrame(0, index=all_labels, columns=all_labels)
    for ls in df.labels:
        for a, b in itertools.product(set(ls), set(ls)):
            M.loc[a, b] += 1
    st.dataframe(M.style.background_gradient(cmap="Blues"), use_container_width=True)
    st.subheader("Korelasi (phi) antarlabel")
    B = pd.DataFrame({l: df.labels.apply(lambda x, l=l: int(l in x)) for l in all_labels})
    st.dataframe(B.corr().round(2).style.background_gradient(cmap="coolwarm", vmin=-1, vmax=1), use_container_width=True)
```

```
# ------------------------------------------------------------- NER
elif page == "NER Analysis":
    ents = pd.DataFrame(
        [
            {"id": r["id"], "label": e["label"], "text": e["text"], "n_tokens": len(e["text"].split()), "n_chars": e["end"] - e[
    "start"], "text_norm": e["text"].lower()}
            for _, r in df.iterrows()
            for e in r["entities"]
        ]
    )
    if ents.empty:
        st.info("Belum ada span pada data ini.")
        st.stop()
    st.subheader("Jumlah span per label")
    st.bar_chart(ents["label"].value_counts().reindex(all_labels).fillna(0))
    c1, c2 = st.columns(2)
    with c1:
        st.subheader("Span per teks")
        st.bar_chart(df["n_entities"].value_counts().sort_index())
    with c2:
        st.subheader("Panjang span (token)")
        st.bar_chart(ents["n_tokens"].clip(upper=15).value_counts().sort_index())
    st.subheader("Dominasi tag O (level token)")
    total_tok = int(df.n_tokens.sum())
    ent_tok = int(ents.n_tokens.sum())
    st.write(f"Token dalam span: {ent_tok} dari {total_tok} ({ent_tok / total_tok:.1%}); sisanya (**{1 - ent_tok / total_tok:.1%
    }**) bertag O. "
             "Perkiraan dihitung dari jumlah kata; span yang tumpang tindih dihitung dua kali.")
    st.subheader("Span paling sering muncul")
    lab = st.selectbox("Label", ["(semua)"] + all_labels)
    e2 = ents if lab == "(semua)" else ents[ents.label == lab]
    top = e2.groupby(["label", "text_norm"]).size().reset_index(name="jumlah").sort_values("jumlah", ascending=False).head(20)
    st.dataframe(top, use_container_width=True, hide_index=True)
    st.subheader("Contoh span yang di-highlight")
    pool = df[df.n_entities > 0]
    if lab != "(semua)":
        pool = pool[pool.entities.apply(lambda es: any(e["label"] == lab for e in es))]
    for _, r in pool.sample(min(8, len(pool)), random_state=1).iterrows() if len(pool) else []:
        st.markdown(render_highlight(r["text"], r["entities"]), unsafe_allow_html=True)
        st.divider()
# ------------------------------------------------------------- Filter
else:
    q = st.text_input("Cari teks (tidak peka huruf besar/kecil)")
    f_labels = st.multiselect("Filter label dokumen", all_labels)
    mode = st.radio("Mode label", ["Mengandung salah satu", "Mengandung semua"], horizontal=True)
    f_ents = st.multiselect("Filter label span", all_labels)
    q_span = st.text_input("Cari teks di dalam span")
    m = pd.Series(True, index=df.index)
    if q:
        m &= df.text.str.contains(q, case=False, regex=False)
    if f_labels:
        m &= df.labels.apply(lambda x: all(l in x for l in f_labels) if mode.endswith("semua") else any(l in x for l in f_labels
    ))
    if f_ents:
        m &= df.entities.apply(lambda es: any(e["label"] in f_ents for e in es))
    if q_span:
        m &= df.entities.apply(lambda es: any(q_span.lower() in e["text"].lower() for e in es))
    res = df[m]
    st.write(f"**{len(res)}** teks cocok")
    for _, r in res.head(50).iterrows():
        with st.container(border=True):
            st.markdown(f"**ID {r['id']}**  |  `{', '.join(r['labels']) or '-'}`")
            st.markdown(render_highlight(r["text"], r["entities"]), unsafe_allow_html=True)
    if len(res) > 50:
        st.caption("Menampilkan 50 hasil pertama.")
```

3. tools/preannotate.py

```
"""Pra-anotasi berbasis aturan (leksikon + ekspansi span) untuk draft label HHH.
Hasilnya DRAFT: wajib ditinjau annotator di Prodigy sebelum dipakai sebagai gold."""
import re, json, sys
W=r"[\w'-]+"
MOD=r"(?:sangat|sering|selalu|tidak|gak|ga|nggak|ngga|tak|belum|bisa|lama|cepat|cepet|cpt|mudah|gampang|susah|sulit|ribet|banget
    |bgt|gagal|jelek|bagus|lemot|banyak|byk|murah|mahal|gratis|free|lancar|simple|simpel|sudah|udah|masih|terlalu|kurang|tolong|
    mohon|sekali|amat|sih|nya)"
CUES={
 "PENGIRIMAN":[r"pengiriman",r"kirim\w*",r"kurir",r"ekspedisi",r"\bjne\b",r"\bjnt\b",r"sicepat",r"gosend",r"grab ?express",r"sam
    pai",r"sampe",r"nyampe",r"tiba",r"diterima",r"packing\w*",r"kemasan",r"paket",r"tracking",r"resi",r"telat",r"terlambat",r"be
    lum datang",r"dikirim"],
 "LAYANAN_PELANGGAN":[r"\bcs\b",r"customer ?(?:service|care)",r"costumer ?service",r"call center",r"seller",r"penjual",r"pelapak
    ",r"admin",r"refund\w*",r"retur\w*",r"komplain\w*",r"pelayanan",r"layanan",r"respon\w*",r"hubungi",r"dibalas",r"balas\w*",r"
    pengembalian",r"cancel",r"batal\w*"],
 "APLIKASI_PERFORMA":[r"error\w*",r"bug\w*",r"lemot",r"lambat",r"\blag(?:s|ging|ged)?\b",r"loading",r"force ?close",r"crash\w*",
    r"hang",r"keluar sendiri",r"tidak bisa (?:dibuka|dibuka|login|masuk)",r"gak bisa (?:dibuka|login|masuk)",r"ga bisa (?:dibuka
    |login|masuk)",r"nge-?lag",r"ringan",r"enteng",r"lancar",r"stabil",r"gangguan",r"maintenance",r"ngehang",r"not responding",r
    "gagal (?:login|masuk)",r"keluar terus",r"eror"],
 "USABILITY":[r"tampilan\w*",r"\bui\b",r"\bux\b",r"menu\w*",r"fitur\w*",r"feature",r"navigasi",r"pencarian",r"\bcari\b",r"search
    ",r"mudah\w*",r"gampang",r"simpel",r"simple",r"sederhana",r"user ?friendly",r"praktis",r"ribet",r"susah",r"sulit",r"bingung"
    ,r"filter",r"kategori\w*",r"desain",r"notifikasi",r"notif\b",r"wishlist"],
 "PEMBAYARAN":[r"bayar\w*",r"pembayaran",r"checkout",r"transaksi",r"saldo",r"e-?wallet",r"\bovo\b",r"gopay",r"\bdana\b",r"shopee
    pay",r"linkaja",r"blipay",r"virtual account",r"\bva\b",r"kartu kredit",r"cicilan",r"cod",r"transfer",r"top ?up",r"klikbca",r
    "klikpay",r"debit",r"dana (?:belum )?(?:kembali|masuk)",r"uang (?:belum )?(?:kembali|masuk)",r"payment"],
 "PROMO_HARGA":[r"promo\w*",r"diskon\w*",r"voucher\w*",r"cashback",r"harga\w*",r"mahal\w*",r"murah\w*",r"gratis(?: ongkir)?",r"f
    ree ?(?:ongkir|shipping)",r"ongkir",r"ongkos kirim",r"potongan",r"hemat",r"flash ?sale",r"harbolnas",r"poin\w*",r"point\w*",
    r"kupon",r"bonus",r"deal"],
}
# kata yang bukan bukti jika berdiri sendiri (menghindari false positive)
WEAK={"PENGIRIMAN":{"sampai"},"LAYANAN_PELANGGAN":{"respon","batal","cancel"},"USABILITY":{"fitur","cari","notif","notifikasi"},
    "PEMBAYARAN":{"transaksi","dana","transfer","cod"}}
def tokens(t):
    return [(m.start(),m.end(),m.group()) for m in re.finditer(r"\S+",t)]
def clean(t,s,e):
    P=" ,.!?;:…\"'()*-"
    while s<e and t[s] in P: s+=1
    while e>s and t[e-1] in P: e-=1
    return s,e
def predict(text):
    low=text.lower(); out=[]
    toks=tokens(text)
    for lab,pats in CUES.items():
        for p in pats:
            for m in re.finditer(p,low):
                if low[m.start():m.end()].strip() in WEAK.get(lab,()):
                    continue
                s,e=m.start(),m.end()
                # perluas ke kata utuh
                while s>0 and re.match(r"[\w]",low[s-1]): s-=1
                while e<len(low) and re.match(r"[\w]",low[e]): e+=1
                # ekspansi satu-dua kata modifier di kiri/kanan dalam klausa
                for _ in range(2):
                    mm=re.search(r"("+MOD+r")\s+$",low[:s])
                    if mm and not re.search(r"[,.!?;]\s*$",low[:s]): s=mm.start()
                    else: break
                for _ in range(2):
                    mm=re.match(r"\s+("+MOD+r")\b",low[e:])
                    if mm and not re.search(r"[,.!?;]",low[e-1:e]): e+=mm.end()
                    else: break
                s,e=clean(text,s,e)
                if e>s: out.append((s,e,lab))
    # gabung span sama-label yang bertumpuk/berdekatan
    out.sort()
    merged=[]
    for s,e,l in out:
        for k,(s2,e2,l2) in enumerate(merged):
            if l==l2 and s<=e2+1 and e>=s2:
                merged[k]=(min(s,s2),max(e,e2),l);break
        else: merged.append((s,e,l))
    return sorted(merged)
def labels(spans): return sorted({l for _,_,l in spans})
if __name__=="__main__":
    print(predict(sys.argv[1]))
```

4. tools/make_prefill.py

```
import json, re, sys, os, csv, collections
sys.path.insert(0,".")
from preannotate import predict, labels
src="../dataset/prodigy_split"; out="../dataset/prodigy_prefill"; os.makedirs(out,exist_ok=True)
def toks(t): return [{"text":m.group(),"start":m.start(),"end":m.end(),"id":i} for i,m in enumerate(re.finditer(r"\S+",t))]
def snap(t,tk,s,e):
    a=next((x["id"] for x in tk if x["end"]>s),None); b=next((x["id"] for x in reversed(tk) if x["start"]<e),None)
    return None if a is None or b is None or b<a else (a,b)
summary={}
for who in ["Dani","Isidorus","Johannes"]:
    rows=[json.loads(l) for l in open(f"{src}/HHH_individu_{who}.jsonl",encoding="utf-8")]
    n_empty=0;ncsv=[]
    with open(f"{out}/HHH_individu_{who}_prefill.jsonl","w",encoding="utf-8") as f:
        for r in rows:
            sp=predict(r["text"]);tk=toks(r["text"]);spans=[]
            for s,e,l in sp:
                ab=snap(r["text"],tk,s,e)
                if ab: spans.append({"start":tk[ab[0]]["start"],"end":tk[ab[1]]["end"],"token_start":ab[0],"token_end":ab[1],"la
    bel":l})
            labs=sorted({s["label"] for s in spans}); n_empty+=not labs
            task={"text":r["text"],"meta":r["meta"],"tokens":tk,"spans":spans,"accept":labs}
            f.write(json.dumps(task,ensure_ascii=False)+"\n")
            ncsv.append([r["meta"]["id"],r["text"],",".join(labs) if labs else "(SARAN: IGNORE)"," | ".join(f'{r["text"][s["star
    t"]:s["end"]]} = {s["label"]}' for s in spans)])
    with open(f"{out}/HHH_individu_{who}_draft_review.csv","w",encoding="utf-8",newline="") as f:
        w=csv.writer(f);w.writerow(["id","text","draft_label","draft_span"]);w.writerows(ncsv)
    summary[who]=(len(rows),n_empty)
    print(who,len(rows),"saran Ignore (tanpa cue):",n_empty)
```

5. analisis/iaa.py (hitung agreement)

```
import json, collections, itertools, sys
import numpy as np
rows=[json.loads(l) for l in open(sys.argv[1],encoding="utf-8")]
L=["PENGIRIMAN","LAYANAN_PELANGGAN","APLIKASI_PERFORMA","USABILITY","PEMBAYARAN","PROMO_HARGA"]
name={"240712771":"Dani","ambar":"Isidorus","Joo":"Johannes"}
by=collections.defaultdict(dict)
for r in rows: by[r["meta"]["id"]][name[r["_annotator_id"]]]=r
A=["Dani","Isidorus","Johannes"]
def kappa(a,b):
    a=np.array(a);b=np.array(b);po=(a==b).mean()
    pe=a.mean()*b.mean()+(1-a.mean())*(1-b.mean())
    return (po-pe)/(1-pe) if pe<1 else float('nan')
full=[i for i,v in by.items() if len(v)==3]
print("items",len(by),"labeled by all 3:",len(full))
print("\nPairwise Cohen kappa per label:", L)
for x,y in itertools.combinations(A,2):
    ids=[i for i,v in by.items() if x in v and y in v]
    out=[round(kappa([l in by[i][x]["accept"] for i in ids],[l in by[i][y]["accept"] for i in ids]),2) for l in L]
    exact=np.mean([set(by[i][x]["accept"])==set(by[i][y]["accept"]) for i in ids])
    print(x,y,len(ids),out,"macro",round(np.nanmean(out),2),"exact-set",round(exact,2))
print("\nFleiss per label (3 raters):")
fl=[]
for l in L:
    M=np.array([[sum(l in by[i][a]["accept"] for a in A),3-sum(l in by[i][a]["accept"] for a in A)] for i in full])
    n=3;P=((M**2).sum(1)-n)/(n*(n-1));Pb=P.mean();p=M.sum(0)/M.sum();Pe=(p**2).sum()
    k=(Pb-Pe)/(1-Pe);fl.append(k);print(" ",l,round(k,2),"prevalence",round(p[0],2))
print("macro Fleiss",round(np.mean(fl),2))
print("all 3 give identical label set",round(np.mean([len({frozenset(by[i][a]["accept"]) for a in A})==1 for i in full]),2))
def spans(r): return {(s["start"],s["end"],s["label"]) for s in (r.get("spans") or [])}
f1=lambda tp,a,b:2*tp/(a+b) if a+b else 0
print("\nSpan agreement pairwise")
for x,y in itertools.combinations(A,2):
    ids=[i for i,v in by.items() if x in v and y in v]
    tp=fx=fy=otp=0
    lx=[];ly=[]
    for i in ids:
        a=spans(by[i][x]);b=spans(by[i][y]);tp+=len(a&b);fx+=len(a);fy+=len(b)
        used=set()
        for s in a:
            for t in b:
                if t not in used and s[2]==t[2] and min(s[1],t[1])>max(s[0],t[0]): otp+=1;used.add(t);break
        for k in range(len(by[i][x]["tokens"])):
            def lab(r):
                for s in (r.get("spans") or []):
                    if s["token_start"]<=k<=s["token_end"]: return s["label"]
                return "O"
            lx.append(lab(by[i][x]));ly.append(lab(by[i][y]))
    lx=np.array(lx);ly=np.array(ly);po=(lx==ly).mean()
    pe=sum((lx==c).mean()*(ly==c).mean() for c in set(lx)|set(ly))
    print(f"  {x}-{y} n={len(ids)} exact F1={f1(tp,fx,fy):.2f} overlap F1={f1(otp,fx,fy):.2f} token kappa={(po-pe)/(1-pe):.2f}")
print()
for a in A:
    rs=[v[a] for v in by.values() if a in v]
    print(a,len(rs),"avg labels/text",round(np.mean([len(r["accept"]) for r in rs]),2),"spans/text",round(np.mean([len(r.get('sp
    ans') or []) for r in rs]),2),dict(collections.Counter(l for r in rs for l in r["accept"])))
print("missing per annotator",{a:len(set(by)-{i for i,v in by.items() if a in v}) for a in A})
# disagreement examples
dis=[i for i in full if len({frozenset(by[i][a]["accept"]) for a in A})>1]
import random;random.seed(1)
print("\nContoh disagreement:")
for i in random.sample(dis,8):
    print(by[i]["Dani"]["text"][:110]);
    for a in A: print("   ",a,sorted(by[i][a]["accept"]))
```

6. analisis/adjudication gold.py

```
import sys, json, collections
sys.path.insert(0,"/home/claude/build")
from adj import *
IGN="IGN"
# Keputusan adjudication manual (mengacu guideline v2.1); selain ini = suara mayoritas >=2 dari 3
O={
 6833:{"PEMBAYARAN"}, 4242:{"USABILITY"}, 4958:{"PENGIRIMAN","PEMBAYARAN"}, 3666:{"LAYANAN_PELANGGAN","PEMBAYARAN"},
 5085:{"PEMBAYARAN","PENGIRIMAN","PROMO_HARGA","USABILITY"}, 670:{"USABILITY"}, 2339:{"APLIKASI_PERFORMA","LAYANAN_PELANGGAN","P
    EMBAYARAN"},
 4117:{"PEMBAYARAN","PENGIRIMAN","PROMO_HARGA"}, 7607:{"PENGIRIMAN","PROMO_HARGA","USABILITY"}, 2041:{"APLIKASI_PERFORMA","PEMBA
    YARAN","USABILITY"},
 4006:{"LAYANAN_PELANGGAN","PEMBAYARAN","PROMO_HARGA"}, 210:{"APLIKASI_PERFORMA","LAYANAN_PELANGGAN","PEMBAYARAN"}, 9957:{"PROMO
    _HARGA"},
 1679:{"LAYANAN_PELANGGAN","PENGIRIMAN","PROMO_HARGA"}, 1698:{"PROMO_HARGA","USABILITY"}, 3020:{"LAYANAN_PELANGGAN","PENGIRIMAN"
    ,"USABILITY"},
 1076:{"PEMBAYARAN","PENGIRIMAN"}, 2053:{"LAYANAN_PELANGGAN","PENGIRIMAN","PROMO_HARGA"}, 9864:{"PENGIRIMAN"}, 1164:{"APLIKASI_P
    ERFORMA","USABILITY"},
 6961:{"PEMBAYARAN","PENGIRIMAN","PROMO_HARGA","USABILITY"}, 1237:{"LAYANAN_PELANGGAN","PROMO_HARGA"}, 4984:{"PROMO_HARGA"}, 348
    1:{"PROMO_HARGA"},
 9839:{"PENGIRIMAN"}, 5789:{"PEMBAYARAN"}, 2029:{"PENGIRIMAN"}, 6688:{"PROMO_HARGA"}, 9777:{"PROMO_HARGA"}, 8837:{"PEMBAYARAN"},
     3492:{"PENGIRIMAN"},
 5692:IGN, 3156:IGN, 2062:IGN, 6706:IGN, 4768:IGN,
}
def iou(a,b):
    i=max(0,min(a[1],b[1])-max(a[0],b[0])); u=max(a[1],b[1])-min(a[0],b[0]); return i/u if u else 0
def final_labels(i):
    if i in O: return None if O[i]==IGN else set(O[i])
    maj,ni,votes=majority(i)
    if ni>=2: return None
    return maj
def spans_for(i,labs):
    out=[];unsup=[]
    for lab in sorted(labs):
        cands=[]
        for a in A:
            if a not in by[i]: continue
            if lab not in by[i][a]["accept"]: continue
            for s in (by[i][a].get("spans") or []):
                if s["label"]==lab: cands.append((s["start"],s["end"],a))
        clusters=[]
        for c in cands:
            for cl in clusters:
                if any(min(c[1],x[1])>max(c[0],x[0]) and c[2]!=x[2] or (c[2]==x[2] and False) for x in cl) or any(min(c[1],x[1])
    >max(c[0],x[0]) for x in cl):
                    cl.append(c);break
            else: clusters.append([c])
        if not clusters: unsup.append(lab);continue
        sup=lambda cl:len({x[2] for x in cl})
        best=max(sup(cl) for cl in clusters)
        keep=[cl for cl in clusters if sup(cl)>=2] or [cl for cl in clusters if sup(cl)==best]
        for cl in keep:
            med=max(cl,key=lambda x:sum(iou(x,y) for y in cl))
            out.append({"start":med[0],"end":med[1],"label":lab,"text":text[i][med[0]:med[1]],"support":sup(cl)})
    return sorted(out,key=lambda s:(s["start"],s["end"])),unsup
gold=[];ign=[];nosp=[]
for i in sorted(by):
    labs=final_labels(i)
    if labs is None: ign.append(i);continue
    sp,uns=spans_for(i,labs)
    if uns: nosp.append((i,uns))
    gold.append({"id":i,"text":text[i],"labels":sorted(labs),"entities":[{k:v for k,v in s.items() if k!="support"} for s in sp]
    ,
                 "adjudication":{"annotators":sorted(by[i]),"ignored_by":sorted(set(A)-set(by[i])),"method":"override" if i in O
     else "majority"}})
print("gold",len(gold),"ignored",len(ign),"labels lacking span",len(nosp))
print(collections.Counter(l for g in gold for l in g["labels"]))
print(collections.Counter(e["label"] for g in gold for e in g["entities"]))
json.dump({"gold":gold,"nosp":nosp,"ign":ign},open("/home/claude/build/gold_tmp.json","w"),ensure_ascii=False)
for i,u in nosp[:20]: print(i,u,text[i][:90])
# --- span manual untuk label tanpa span dari annotator (keputusan adjudication) ---
M={(210,"LAYANAN_PELANGGAN"):"direfund makan waktu 1-3hr",(387,"PROMO_HARGA"):"murah",(834,"PEMBAYARAN"):"metode pembayaran tida
    k ada",
 (1164,"APLIKASI_PERFORMA"):"lancar",(1164,"USABILITY"):"mudah",(1237,"PROMO_HARGA"):"barang mahal2",(3020,"LAYANAN_PELANGGAN"):
    "Komplain juga susah",
 (4242,"USABILITY"):"ngga susah cara berbelanjanya",(4311,"USABILITY"):"semua orang bisa memberi review tanpa benar2 membeli",
 (4958,"PEMBAYARAN"):"sudah dibayar juga di toko alfamart",(4958,"PENGIRIMAN"):"barangnya mau di ambil di toko yang dituju kenap
    a gak bisa",
 (6833,"PEMBAYARAN"):"udah bayar eh nggk terbatalkan padahal sudah bayar",(6961,"PROMO_HARGA"):"pengiriman gratis",(9590,"PROMO_
    HARGA"):"promo",
 (9705,"LAYANAN_PELANGGAN"):"ditanya ke CS 4 kali"}
for g in gold:
    for (i,lab),sub in M.items():
        if g["id"]==i and lab in g["labels"] and not any(e["label"]==lab for e in g["entities"]):
            s=g["text"].find(sub); assert s>=0,(i,sub)
            g["entities"].append({"start":s,"end":s+len(sub),"label":lab,"text":sub})
    P=" ,.!?;:…\"'()*-"
    for e in g["entities"]:
```

```
        t=g["text"]; st,en=e["start"],e["end"]
        while st<en and t[st] in P: st+=1
        while en>st and t[en-1] in P: en-=1
        e["start"],e["end"],e["text"]=st,en,t[st:en]
    g["entities"]=[e for e in g["entities"] if e["end"]>e["start"]]
    g["entities"].sort(key=lambda e:(e["start"],e["end"]))
    for e in g["entities"]: assert g["text"][e["start"]:e["end"]]==e["text"]
    assert {e["label"] for e in g["entities"]}<=set(g["labels"])
    miss=set(g["labels"])-{e["label"] for e in g["entities"]}
    assert not miss,(g["id"],miss)
import os
os.makedirs("/home/claude/proj/dataset",exist_ok=True)
with open("/home/claude/proj/dataset/KelpHHH_gold_overlap.jsonl","w",encoding="utf-8") as f:
    for g in gold: f.write(json.dumps(g,ensure_ascii=False)+"\n")
# tabel adjudication (csv) untuk lampiran laporan
import csv
with open("/home/claude/proj/dataset/KelpHHH_adjudication_log.csv","w",encoding="utf-8",newline="") as f:
    w=csv.writer(f);w.writerow(["id","text","Dani","Isidorus","Johannes","final","metode"])
    for i in sorted(by):
        fl=final_labels(i)
        row=[i,text[i]]+[(",".join(sorted(by[i][a]["accept"])) if a in by[i] else "IGNORE") for a in A]+[("IGNORE" if fl is None
     else ",".join(sorted(fl))),"override" if i in O else "majority"]
        w.writerow(row)
print("ok gold file",len(gold),"all spans validated")
```

7. analisis/merge_final.py (gabung gold overlap + individu)

```
"""Gabungkan gold overlap + anotasi individu (ekspor Prodigy) -> KelpHHH_dataset_annotation.jsonl
Jalankan dari folder analysis/:  python merge_final.py
Ekspor individu: ../dataset/prodigy_export/individu_<Nama>.jsonl (Dani, Isidorus, Johannes)."""
import json, glob, os, collections
P=" ,.!?;:…\"'()*-"
out=[]; stats=collections.Counter()
for l in open("../dataset/KelpHHH_gold_overlap.jsonl",encoding="utf-8"):
    g=json.loads(l); g["source"]="overlap_adjudicated"; out.append(g); stats["overlap"]+=1
seen={g["id"] for g in out}
name={"240712771":"Dani","ambar":"Isidorus","Joo":"Johannes"}
for f in sorted(glob.glob("../dataset/prodigy_export/individu_*.jsonl")):
    who=os.path.basename(f)[8:-6]
    for l in open(f,encoding="utf-8"):
        r=json.loads(l); i=r["meta"]["id"]
        if i in seen: stats["duplikat_dilewati"]+=1; continue
        if r.get("answer")!="accept": stats["bukan_accept_dibuang_"+who]+=1; continue
        labs=sorted(set(r.get("accept",[])))
        extra={s["label"] for s in (r.get("spans") or [])}-set(labs)
        if extra: labs=sorted(set(labs)|extra); stats["label_ditambah_dari_span_"+who]+=1
        if not labs: stats["tanpa_label_dibuang_"+who]+=1; continue
        ents=[]
        for s in r.get("spans") or []:
            st,en,t=s["start"],s["end"],r["text"]
            while st<en and t[st] in P: st+=1
            while en>st and t[en-1] in P: en-=1
            if en>st: ents.append({"start":st,"end":en,"label":s["label"],"text":t[st:en]})
        # Koreksi sistematis: draft pra-anotasi v1 keliru menandai "lagi" (= again) sebagai APLIKASI_PERFORMA
        import re
        n0=len(ents)
        ents=[e for e in ents if not (e["label"]=="APLIKASI_PERFORMA" and re.fullmatch(r"(?i)(lagi|lagu|lagian|laga)",e["text"])
    )]
        if len(ents)<n0:
            stats["span_lagi_dikoreksi_"+who]+=n0-len(ents)
            if "APLIKASI_PERFORMA" in labs and not any(e["label"]=="APLIKASI_PERFORMA" for e in ents):
                labs=[l for l in labs if l!="APLIKASI_PERFORMA"]; stats["label_APLIKASI_dihapus_"+who]+=1
            if not labs: stats["tanpa_label_setelah_koreksi_"+who]+=1; continue
        ents.sort(key=lambda e:(e["start"],e["end"]))
        for e in ents: assert r["text"][e["start"]:e["end"]]==e["text"]
        if set(labs)-{e["label"] for e in ents}: stats["label_tanpa_span_"+who]+=1
        out.append({"id":i,"text":r["text"],"labels":labs,"entities":ents,"source":"individu:"+who}); seen.add(i); stats["indivi
    du_"+who]+=1
with open("../dataset/KelpHHH_dataset_annotation.jsonl","w",encoding="utf-8") as f:
    for g in out: f.write(json.dumps(g,ensure_ascii=False)+"\n")
print("total",len(out),dict(stats))
```